SpecialistRoute Handover — 2. Environment Setup

Oct 25, 2026 · @Anas Alsirafy

Everything runs locally on the AUC workstation in the Python virtual environment `/home/auc/tf_env` (TensorFlow with Keras 3, local GPU). No cloud service or paid compute is needed.

Machine and access

- **Workstation:** the AUC lab machine, Linux (Ubuntu under WSL), worked on through VS Code. All project files live under `/home/auc/PlantDoc/`.
- **GPU:** one local GPU with no usage limit. Every training run, the 5-seed sweeps and the ensemble caching ran on it.
- **If WSL stops responding** or VS Code shows a remote-connection error (code 1006): run `wsl --shutdown` in Windows PowerShell, reopen the WSL terminal, then reconnect VS Code. Nothing on disk is lost.

Python environment

The project uses the virtual environment `/home/auc/tf_env`. The system Python (3.12) is "externally managed" and refuses `pip install`; never use `--break-system-packages` on it.

- **Notebook kernel:** select `tf_env` in Jupyter / VS Code. Check with `import sys; print(sys.executable)`, which must print `/home/auc/tf_env/bin/python`.
- **Installing a package from a notebook:** `%pip install <package>`, then restart the kernel. `%pip` always installs into the kernel's own environment.
- **Installing from a terminal:** `/home/auc/tf_env/bin/python -m pip install <package>`, or `source /home/auc/tf_env/bin/activate` first.
- **Subprocesses use the same Python.** The OOM-safe caching and feature-extraction workers start with `sys.executable`, so anything importable in the notebook is importable in the workers.
- **Open item:** exact package versions were never frozen. Before changing anything, run `/home/auc/tf_env/bin/python -m pip freeze > /home/auc/PlantDoc/requirements_v2.txt` and keep that file.

Packages

Package	Used for	Notes
tensorflow (Keras 3)	All models, data loading, training	`KERAS_BACKEND=tensorflow` is set in Main section 1
keras_cv	RandAugment layer	Imported in Main section 1, so it must be installed even though RandAugment is off
numpy, pandas, scipy	Arrays, tables, t-intervals	
scikit-learn	Metrics, StratifiedGroupKFold, logistic regression	
opencv-python	CLAHE, image-quality metrics	
pillow	Contact sheets, image I/O in data prep	Never used for model inputs (see Known Issues)
imagehash	Perceptual-hash duplicate detection	Data prep only
cleanlab	Label-quality scores	Label-quality notebook; installed into `tf_env`
matplotlib, seaborn	Plots	
openpyxl	Writing `v2_all_results.xlsx`	Used by `collect_v2_results.py`

Pretrained weights and directory layout

ImageNet weights are loaded from local files, so no internet access is needed. Their paths are listed in `MODEL_PATHS` (Main section 2):

- `/home/auc/PlantDoc/Models/convnext_base_notop.h5`
- `/home/auc/PlantDoc/Models/convnext_large_notop.h5`
- `/home/auc/PlantDoc/Models/efficientnet-b0_noisy-student_notop.h5`: does **not** load under Keras 3; no longer used (see Known Issues)

Other entries in `MODEL_PATHS` (MobileNet, EfficientNet-B3/V2, NASNet, v1 checkpoints) point to `./plantdoc_local/...` and belong to legacy v1 experiments. Setting an entry to `"imagenet"` makes Keras download the weights instead, which needs internet.

```
/home/auc/PlantDoc/
├── Datasets/
|   ├── dataset/{train,val,test}/   final v2 data (output of
prepare_final_dataset)
|   ├── reports/                    manifest, exclusion list, duplicate
review, counts
|   └── derived_v2/                 specialist and classifier folders, built
automatically
├── Experiments/
|   ├── v2_<Component>/             checkpoints, histories, progress.json,
config.txt
|   ├── v2_pipeline/                pipeline logs, ensemble caches (probs/),
summaries
|   ├── v2_label_quality/           label-quality outputs and review sheets
|   ├── v2_superseded/              runs archived after a preset changed
|   ├── v2_run_all_state.json       orchestrator state
|   └── v2_all_results.xlsx         collected results
└── Models/                         ImageNet no-top weights
```

Open item: confirm where the notebooks themselves are stored (expected next to `Datasets/` and `Experiments/`). The label-quality and data-prep notebooks need their path settings checked against this layout.

First-run checklist

These checks confirm the environment, data and saved results without training anything.

- [ ] Open `PlantDoc_Main.ipynb` with the `tf_env` kernel and run section 1. It prints the TensorFlow version and `GPUs available: 1` (or more).
- [ ] Run sections 2–3, then `check_protocol()`. Expected: train 1,897 / val 209 / test 236 images, "no identical image in more than one split", `RESULT: OK`.
- [ ] Run sections 4–8, then `evaluate_pipeline(42)`. Expected: **176/236 = 74.58%** final accuracy and 223/236 routing accuracy, matching the saved seed-42 result.
- [ ] Optional: run 11a, then `check_matches_section8([42, 123, 2024, 779, 1027])`. Every seed must print OK (cached ensemble predictions match the per-seed evaluations).

If the seed-42 number differs, stop and check the image loader first (Known Issues, "PIL vs TensorFlow loader").